ACTIVIDAD LABORATORIO NO.1
CREACIÓN DE ESPECIFICACIONES LÉXICAS PARA TOKENS DETERMINADOS

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING ROGERIO ORLANDO BELTRAN CASTRO

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
02 DE MARZO
2026

TABLA DE CONTENIDO

TABLA DE CONTENIDO **2**
INTRODUCCIÓN **3**
DESARROLLO ACTIVIDAD **3**
1. Marco Teórico 4
1.1 Análisis Léxico 4
1.2 Token vs Lexema 5
1.3 Expresiones Regulares 5
1.4 Autómatas Finitos Deterministas (DFA) 6
1.5 JFlex como Generador Léxico 7
2. Definición Formal de Tokens 8
3. Tabla de Símbolos 10
3.1 Clase Simbolo 10
3.2 Clase TablaSimbolos 10
3.3 Función dentro del Analizador 10
3.4 Importancia de la Tabla de Símbolos 11
4. Arquitectura del Proyecto 11
4.1 Paquete código 12
4.1.1 Tokens.java 12
4.1.2 Simbolo.java 12
4.1.3 TablaSimbolos.java 12
4.1.4 FCompilador.java 14
4.1.5 VentanaAnalizador.java 14
4.2 Carpeta Other Sources / jflex 14
4.2.1 Definition.flex 14
4.3 Carpeta Generated Sources (jflex) 14
4.3.1 Lexer.java 14
4.4 Archivo pom.xml 14
5. Implementación 14
5.1 Directivas 15
5.2 Definiciones (Macros) 15
5.3 Reglas Léxicas 15
5.4 Manejo de Errores Léxicos 17
5.5 Integración con Maven 17
5.6 Integración con la Interfaz Gráfica 17
6. Ejecución y Resultados 17
6.1 Código Analizado 18
6.2 Resultados del Análisis 18
6.3 Detección de Errores Léxicos 19
6.4 Validación del Funcionamiento 19
CONCLUSIONES DE LA ACTIVIDAD **20**
BIBLIOGRAFÍA **20**
AGRADECIMIENTO **21**

INTRODUCCIÓN

A continuación, se presenta el desarrollo paso a paso del laboratorio orientado a la implementación de un analizador léxico para un subconjunto del lenguaje Java. El proceso fue abordado de manera estructurada, partiendo de la definición formal de las categorías léxicas hasta la construcción de una aplicación funcional con interfaz gráfica que permite analizar código fuente y visualizar los tokens identificados.

En primer lugar, se procedió a la definición formal de los tokens del lenguaje, estableciendo mediante expresiones regulares las reglas correspondientes a palabras reservadas, identificadores, literales numéricos (enteros y decimales), cadenas de texto, operadores aritméticos, relacionales y lógicos, separadores, comentarios y símbolos no reconocidos. Estas reglas fueron especificadas en el archivo Definition.flex, el cual constituye la base formal del analizador léxico.

Posteriormente, se utilizó la herramienta JFlex para generar automáticamente el archivo Lexer.java, el cual implementa internamente un Autómata Finito Determinista (DFA) capaz de reconocer los patrones definidos. La generación del analizador se integró al ciclo de construcción del proyecto mediante el plugin jflex-maven-plugin, permitiendo que el código del lexer se actualice automáticamente durante la compilación con Maven.

Una vez generado el analizador léxico, se implementó una estructura de tabla de símbolos compuesta por las clases Simbolo y TablaSimbolos. Esta estructura permite almacenar cada token identificado junto con su lexema, tipo, línea y columna dentro del código fuente, facilitando el seguimiento preciso de la información léxica y el manejo de errores.

En la siguiente etapa se desarrolló una interfaz gráfica utilizando la biblioteca Swing de Java. La aplicación presenta dos áreas de texto: una para el ingreso del código fuente y otra para la visualización de la tabla de símbolos generada. Además, se incorporaron controles de interacción como los botones “Analizar” y “Limpiar”, permitiendo ejecutar el proceso de análisis de manera dinámica. Esta interfaz transforma el analizador en una herramienta interactiva, mejorando la comprensión del proceso léxico.

Durante la ejecución del sistema, el programa recorre el texto ingresado carácter por carácter, identifica los lexemas conforme a las reglas definidas y registra cada token en la tabla de símbolos. En caso de encontrar caracteres no válidos dentro del lenguaje definido, el sistema los clasifica como errores léxicos sin interrumpir el análisis, lo que permite reportar múltiples errores en una sola ejecución.

Cada fase del laboratorio fue validada mediante pruebas con fragmentos de código que incluyen declaraciones de variables, estructuras condicionales, operadores compuestos y errores intencionales. Los resultados se visualizaron directamente en la interfaz gráfica, mostrando la correcta clasificación de los tokens y la detección precisa de errores.

Este enfoque progresivo permitió consolidar la comprensión práctica del análisis léxico, evidenciando cómo los fundamentos teóricos de los autómatas y las expresiones regulares se traducen en una implementación funcional dentro de un entorno profesional de desarrollo.

DESARROLLO ACTIVIDAD

A continuación, se presenta el desarrollo paso a paso del laboratorio orientado a la implementación de un analizador léxico para un subconjunto del lenguaje Java, utilizando la herramienta JFlex como generador automático de analizadores léxicos y el entorno NetBeans con Maven para la gestión y construcción del proyecto. El proceso fue abordado de manera estructurada, construyendo el sistema progresivamente desde la definición formal de los tokens hasta la implementación de una interfaz gráfica interactiva que permite visualizar los resultados del análisis.

En primer lugar, se procedió a la definición formal de las categorías léxicas del lenguaje. Para ello, se establecieron expresiones regulares que describen cada tipo de token, incluyendo palabras reservadas, identificadores, números enteros y decimales, cadenas de texto, operadores aritméticos y relacionales, separadores, comentarios y símbolos no reconocidos. Estas definiciones fueron consignadas en el archivo Definition.flex, el cual constituye la especificación léxica del sistema.

Posteriormente, se utilizó JFlex para transformar dicha especificación en código Java ejecutable, generando automáticamente el archivo Lexer.java. Este archivo implementa internamente un Autómata Finito Determinista (DFA) encargado de reconocer los patrones definidos y clasificar cada lexema conforme a las reglas establecidas. La generación del analizador se integró al ciclo de construcción mediante el plugin jflex-maven-plugin, lo que permitió automatizar el proceso durante la compilación del proyecto.

Una vez generado el analizador léxico, se implementó una tabla de símbolos para almacenar la información resultante del proceso de análisis. Cada token identificado se registra junto con su lexema, tipo, línea y columna dentro del código fuente. Esta estructura permite realizar un seguimiento detallado de los elementos reconocidos y facilita la detección de errores léxicos.

Posteriormente, se desarrolló una interfaz gráfica utilizando la biblioteca Swing de Java. La aplicación presenta un área de entrada para el código fuente y un área de salida donde se muestra la tabla de símbolos generada. Mediante los botones de control, el usuario puede ejecutar el análisis y limpiar la información visualizada. Esta interfaz convierte el sistema en una herramienta interactiva que permite observar de manera clara el funcionamiento del proceso léxico.

Finalmente, se realizaron pruebas con fragmentos de código que incluyen declaraciones de variables, estructuras condicionales, operadores compuestos y caracteres inválidos introducidos de forma intencional. Cada ejecución permitió verificar la correcta clasificación de los tokens y la adecuada identificación de errores, garantizando la coherencia entre la especificación formal y el comportamiento del analizador.

Cada etapa del laboratorio fue validada mediante la revisión de los resultados obtenidos en la interfaz gráfica, asegurando la correcta implementación del sistema y la fidelidad del reconocimiento léxico. Este enfoque progresivo permitió consolidar la comprensión práctica del análisis léxico y evidenciar cómo los principios teóricos de los autómatas finitos y las expresiones regulares se materializan en una aplicación funcional dentro del ámbito de la ingeniería de software.

1. Marco Teórico

1.1 Análisis Léxico

El análisis léxico constituye la primera fase dentro del proceso de compilación de un lenguaje de programación. Su función principal es transformar una secuencia lineal de caracteres provenientes del código fuente en una secuencia estructurada de unidades léxicas denominadas tokens.

Un token representa una categoría sintáctica elemental del lenguaje, como palabras reservadas, identificadores, literales numéricos, operadores o separadores. Cada token está compuesto por dos elementos fundamentales:

- El tipo de token, que indica la categoría léxica reconocida.
- El lexema, que corresponde a la cadena exacta de caracteres encontrada en el código fuente.

El analizador léxico, también conocido como scanner o lexer, recorre el código carácter por carácter y agrupa los símbolos conforme a reglas previamente definidas mediante expresiones regulares. Estas reglas describen formalmente los patrones válidos del lenguaje y permiten identificar de manera determinista cada componente léxico.

Desde el punto de vista formal, el análisis léxico se fundamenta en la teoría de autómatas finitos. Las expresiones regulares que describen los tokens pueden transformarse en Autómatas Finitos Deterministas (DFA), los cuales permiten reconocer patrones de forma eficiente y con complejidad lineal respecto al tamaño del texto de entrada.

En la práctica, herramientas como JFlex automatizan este proceso: a partir de una especificación léxica escrita en un archivo .flex, generan automáticamente el código Java correspondiente al autómata determinista encargado del reconocimiento de tokens.

En el presente laboratorio, el analizador léxico implementado reconoce múltiples categorías léxicas del lenguaje Java, incluyendo palabras reservadas, identificadores, literales, operadores, separadores, comentarios y símbolos no válidos, permitiendo además el registro detallado de cada token junto con su posición dentro del código fuente.

1.2 Token vs Lexema

Dentro del proceso de análisis léxico es fundamental distinguir entre los conceptos de token, lexema y patrón, ya que constituyen los elementos básicos del reconocimiento léxico.

Un token es la categoría léxica asignada a una secuencia de caracteres que cumple una regla específica del lenguaje. Representa el tipo abstracto de unidad reconocida por el analizador. Por ejemplo, categorías como RESERVADA, IDENTIFICADOR, ENTERO, DECIMAL u OP_ARITMETICO corresponden a distintos tipos de tokens definidos en la especificación léxica del sistema.

El lexema, en cambio, es la cadena concreta de caracteres tomada directamente del código fuente que coincide con un patrón determinado. Es decir, el lexema es la instancia específica del token dentro del programa analizado. Por ejemplo:

- En la instrucción int edad = 20;
    - int es un lexema cuyo token es RESERVADA.
    - edad es un lexema cuyo token es IDENTIFICADOR.
    - 20 es un lexema cuyo token es ENTERO.

Por otra parte, el patrón corresponde a la expresión regular que describe formalmente la estructura que debe cumplir un lexema para ser clasificado dentro de un determinado tipo de token. En el caso de los identificadores, el patrón puede definirse como:

$$[a - zA - Z_][a - zA - Z0 - 9_] *$$

Esta expresión indica que un identificador debe comenzar con una letra o guion bajo, seguido opcionalmente por letras, números o guiones bajos.

La relación entre estos tres conceptos puede resumirse de la siguiente manera:

Concepto	Definición	Ejemplo
Token	Categoría léxica asignada por el analizador	IDENTIFICADOR, RESERVADA
Lexema	Cadena real encontrada en el código fuente	int, edad, 1500.50
Patrón	Expresión regular que define la forma del token	[a-zA-Z_][a-zA-Z0-9_]*

En el laboratorio desarrollado, cada lexema reconocido por el analizador es clasificado según su token correspondiente y registrado en la tabla de símbolos junto con su posición dentro del código fuente. Esta distinción permite separar la estructura formal del lenguaje de las instancias concretas que aparecen en un programa determinado, facilitando así el procesamiento posterior en las fases sintáctica y semántica del compilador.

1.3 Expresiones Regulares

Las expresiones regulares son una herramienta formal utilizada para describir patrones dentro de cadenas de texto. En el contexto del análisis léxico, permiten definir de manera precisa la estructura que deben cumplir los lexemas para ser reconocidos como tokens válidos dentro de un lenguaje de programación.

Desde el punto de vista teórico, las expresiones regulares forman parte del estudio de los lenguajes formales y están directamente relacionadas con los lenguajes regulares, los cuales pueden ser reconocidos por autómatas finitos. Esto significa que cualquier patrón descrito mediante una expresión regular puede transformarse en un Autómata Finito Determinista (DFA) equivalente.

Una expresión regular está compuesta por símbolos y operadores que permiten construir patrones complejos a partir de elementos básicos. Entre los operadores más importantes se encuentran:

- Concatenación: unión directa de símbolos (ejemplo: ab)
- Unión (|): alternativa entre patrones (ejemplo: int|float)
- Cerradura de Kleene (*): repetición cero o más veces
- Cerradura positiva (+): repetición una o más veces
- Clases de caracteres []: conjunto de símbolos permitidos
- Agrupación (): organización de subexpresiones

En el laboratorio desarrollado, las expresiones regulares fueron utilizadas para definir formalmente cada categoría léxica del subconjunto del lenguaje Java implementado. Algunos ejemplos relevantes son:

Identificadores:

$$[\mathrm{a}-\mathrm{zA}-Z_][\mathrm{a}-\mathrm{zA}-\mathrm{Z0}-9_]*$$

Números enteros:

$$[0-9]+$$

Números decimales:

$$[0-9]+.[0-9]+$$

Palabras reservadas:

$$\mathrm{int|float|if|else|while|public|class|static|void}$$

Estas expresiones fueron declaradas en el archivo Definition.flex, donde JFlex las interpreta y las transforma automáticamente en un autómata determinista optimizado para el reconocimiento eficiente de tokens.

El uso de expresiones regulares aporta múltiples ventajas:

- Permiten una especificación formal y clara del lenguaje.
- Facilitan la automatización del proceso de generación del analizador léxico.
- Garantizan eficiencia en el reconocimiento de patrones.
- Separan la definición teórica del lenguaje de su implementación concreta.

En conclusión, las expresiones regulares constituyen el puente entre la definición formal de los componentes léxicos y la construcción práctica del analizador, permitiendo convertir reglas abstractas en un sistema funcional capaz de procesar código fuente de manera determinista y eficiente.

1.4 Autómatas Finitos Deterministas (DFA)

Los Autómatas Finitos Deterministas (Deterministic Finite Automata – DFA) son modelos matemáticos utilizados para reconocer lenguajes regulares. Constituyen la base formal sobre la cual se construyen los analizadores léxicos, ya que permiten determinar de manera sistemática si una secuencia de caracteres pertenece a un lenguaje definido mediante expresiones regulares.

Un DFA está compuesto formalmente por una quíntupla:

$$\mathrm{A}=(\mathrm{Q},\Sigma,\delta,\mathrm{q0},\mathrm{F})$$

Donde:

- Q es un conjunto finito de estados.
- Σ (Sigma) es el alfabeto de entrada.

- δ es la función de transición que define el paso de un estado a otro.
- q_0 es el estado inicial.
- F es el conjunto de estados finales o de aceptación.

La característica principal que distingue a un DFA es el determinismo: para cada estado y cada símbolo del alfabeto existe exactamente una transición posible. Esto garantiza que el reconocimiento de una cadena se realice sin ambigüedades y en tiempo lineal respecto a la longitud de la entrada.

En el contexto del análisis léxico, cada token definido mediante una expresión regular puede transformarse en un Autómata Finito No Determinista (NFA), el cual posteriormente se convierte en un DFA equivalente mediante algoritmos formales de determinización. Finalmente, el DFA puede optimizarse mediante procesos de minimización de estados.

Este procedimiento es precisamente el que realizan herramientas como JFlex de manera automática. A partir del archivo Definition.flex, que contiene las expresiones regulares que describen los tokens del lenguaje, JFlex:

- Construye un NFA para cada expresión regular.
- Convierte el NFA en un DFA equivalente.
- Minimiza el número de estados del autómata.
- Genera el código Java correspondiente al autómata resultante.

Durante la ejecución del analizador léxico implementado en este laboratorio, el DFA recorre el texto carácter por carácter, transitando entre estados hasta alcanzar un estado de aceptación que determina el reconocimiento de un token válido. Si no existe una transición válida para un símbolo determinado, el sistema clasifica la secuencia como un error léxico.

El uso de DFA en el análisis léxico presenta múltiples ventajas:

- Garantiza reconocimiento determinista y eficiente.
- Opera en tiempo lineal O(n).
- Permite una implementación directa en código ejecutable.
- Facilita la automatización mediante generadores léxicos.

En conclusión, los Autómatas Finitos Deterministas constituyen el fundamento teórico que permite transformar expresiones regulares en mecanismos computacionales eficientes, haciendo posible la construcción formal y práctica de analizadores léxicos como el desarrollado en el presente laboratorio.

1.5 JFlex como Generador Léxico

JFlex es una herramienta generadora de analizadores léxicos para Java basada en la teoría de lenguajes regulares y autómatas finitos. Su función principal es transformar un conjunto de expresiones regulares definidas en un archivo de especificación (.flex) en código Java que implementa un Autómata Finito Determinista (DFA) optimizado para el reconocimiento de tokens.

En términos prácticos, JFlex automatiza el proceso que, desde el punto de vista teórico, implica:

- Definir los patrones léxicos mediante expresiones regulares.
- Construir un Autómata Finito No Determinista (NFA).
- Convertir el NFA en un DFA equivalente.
- Minimizar el número de estados del autómata.
- Generar el código fuente correspondiente en Java.

De esta manera, JFlex elimina la necesidad de implementar manualmente la lógica de transición de estados, reduciendo errores y aumentando la eficiencia del desarrollo.

En el presente laboratorio, se utilizó JFlex para generar el archivo Lexer.java a partir de la especificación contenida en Definition.flex. Este archivo .flex se organiza en tres secciones principales:

- Directivas y configuraciones, donde se define el nombre de la clase generada, el tipo de retorno de los tokens y opciones como soporte Unicode, conteo de líneas y columnas.
- Definiciones o macros, que contienen abreviaciones para expresiones regulares reutilizables.
- Reglas léxicas, que establecen la correspondencia entre cada patrón y la acción asociada (retorno de un tipo de token).

La integración de JFlex dentro del proyecto se realizó mediante Maven, utilizando el plugin jflex-maven-plugin. Esta configuración permite que el archivo Lexer.java se genere automáticamente durante el proceso de compilación, garantizando que cualquier modificación en la especificación léxica se refleje inmediatamente en el analizador sin intervención manual.

El uso de JFlex presenta múltiples ventajas en el desarrollo de compiladores y herramientas de procesamiento de lenguajes:

- Automatiza la construcción del DFA.
- Produce código optimizado y eficiente.
- Reduce la complejidad de implementación manual.
- Facilita la mantenibilidad del proyecto.
- Es ampliamente utilizado en entornos académicos e industriales.

Gracias a JFlex, el laboratorio no solo implementa los fundamentos teóricos del análisis léxico, sino que también adopta prácticas profesionales de automatización y generación de código, alineadas con estándares modernos de desarrollo de software.

2. Definición Formal de Tokens

En esta sección se presenta la especificación formal de los tokens reconocidos por el analizador léxico desarrollado en el laboratorio. La definición formal constituye el núcleo teórico del sistema, ya que establece con precisión las reglas que permiten identificar y clasificar cada componente léxico dentro del código fuente.

Cada token ha sido definido mediante una expresión regular, la cual describe el patrón formal que debe cumplir una secuencia de caracteres (lexema) para ser reconocida como válida dentro del lenguaje. Estas expresiones regulares representan lenguajes formales que pueden ser reconocidos por Autómatas Finitos Deterministas (DFA), garantizando un proceso de análisis eficiente, determinista y libre de ambigüedades.

La formalización de los tokens no solo permite distinguir categorías como palabras reservadas, identificadores, números o operadores, sino que además asegura coherencia entre la especificación teórica y la implementación práctica en el archivo Definition.flex. De esta manera, cada patrón definido se traduce automáticamente, mediante JFlex, en transiciones dentro del autómata que procesa el texto carácter por carácter.

Es importante destacar que la definición de tokens incluye tanto elementos válidos del lenguaje como una regla explícita para el manejo de errores léxicos. Esto permite que cualquier carácter que no coincida con los patrones establecidos sea clasificado como ERROR, garantizando robustez en el análisis y continuidad en la ejecución. La tabla que se presenta a continuación resume las categorías léxicas implementadas en el laboratorio, indicando para cada token:

- Su nombre dentro del sistema.
- La expresión regular que lo define formalmente.
- Una descripción funcional de su propósito dentro del lenguaje.

Debido a la amplitud de categorías léxicas incluidas —especialmente en el caso de las palabras reservadas del lenguaje Java y los distintos tipos de operadores— la tabla puede extenderse a más de una página. Esta extensión no representa un error de formato, sino que responde a la necesidad de documentar de manera completa y rigurosa la especificación léxica del sistema, asegurando claridad y trazabilidad académica.

La correcta definición formal de estos tokens garantiza:

- Coherencia entre teoría e implementación.
- Reconocimiento determinista mediante DFA.
- Manejo estructurado de errores léxicos.
- Base sólida para futuras fases del compilador (análisis sintáctico y semántico).

A continuación, se presenta la tabla consolidada de tokens definidos en el analizador léxico desarrollado.

Token	Expresión Regular	Descripción
RESERVADA	abstract | assert | boolean | break | byte | case | catch | char | class | const | continue | default | do | double | else | enum | extends | final | finally | float | for | goto | if | implements | import | instanceof | int | interface | long | native | new | package | private | protected | public | return | short | static | strictfp | super | switch | synchronized | this | throw | throws | transient | try | void | volatile | while	Palabras reservadas del lenguaje Java
IDENTIFICADOR	[a-zA-Z_][a-zA-Z0-9_]*	Nombres de variables, clases o métodos
ENTERO	[0-9]+	Números enteros positivos
DECIMAL	[0-9]+\.[0-9]+	Números con parte decimal
CADENA	`"([^"\]	\.)*"`
CARACTER	`'([^'\]	\.)'`
OP_ARITMETICO	\+ | - | * | / | %	Operadores matemáticos
OP_RELACIONAL	== | != | >= | <= | > | <	Operadores de comparación
OP_LOGICO	&& | || | !	Operadores lógicos AND, OR y NOT
OP_ASIGNACION	=	Asignación simple
OP_INCREMENTO	\+\+	Operador de incremento
OP_DECREMENTO	--	Operador de decremento
OP_ASIG_COMPUESTO	\+= | -= | *= | /= | %=	Asignación combinada
SEPARADOR	\(| \) | \{ | \} | \[| \] | ; | , | \. | :	Delimitadores del lenguaje
COMENTARIO_LINEA	//.*	Comentarios de una sola línea
COMENTARIO_BLOQUE	`/*([^*]	*+[^/])*+/`
BOOLEANO	true | false	Literales booleanos
NULO	null	Referencia nula
ERROR	.	Carácter no reconocido

Cada una de estas definiciones fue implementada en el archivo Definition.flex. Durante la ejecución del analizador, el DFA generado por JFlex evalúa el texto de entrada y clasifica cada lexema conforme a estas reglas.

La correcta definición formal de los tokens garantiza:

- Coherencia entre teoría e implementación.
- Reconocimiento determinista.
- Manejo controlado de errores léxicos.
- Base sólida para futuras fases (análisis sintáctico y semántico).

3. Tabla de Símbolos

La tabla de símbolos es una estructura de datos utilizada en los compiladores para almacenar información relevante sobre los elementos identificados durante el proceso de análisis. En el contexto del análisis léxico, su función principal es registrar cada token reconocido junto con información asociada que permita su trazabilidad dentro del código fuente.

En el laboratorio desarrollado, la tabla de símbolos se implementó mediante dos clases principales:

- Simbolo
- TablaSimbolos

3.1 Clase Simbolo

La clase Simbolo representa una entrada individual dentro de la tabla. Cada objeto almacena:

- Lexema: cadena exacta encontrada en el código fuente.
- Tipo de token: categoría léxica asignada.
- Línea: número de línea donde aparece el lexema.
- Columna: posición exacta dentro de la línea.

Esta estructura permite identificar con precisión la ubicación de cada componente léxico, lo cual es fundamental en fases posteriores del compilador, especialmente en la detección y reporte de errores.

3.2 Clase TablaSimbolos

La clase TablaSimbolos gestiona el almacenamiento dinámico de los símbolos identificados durante la ejecución del analizador. Internamente utiliza una lista que almacena todos los objetos Simbolo generados por el lexer.

Además, mantiene una lista independiente para los errores léxicos detectados, lo que permite:

- Registrar todos los errores sin detener la ejecución.
- Reportar múltiples errores en una sola corrida.
- Presentar un resumen final de tokens y errores encontrados.

Durante el proceso de análisis, cada vez que el lexer reconoce un token válido o un error, se crea un objeto Simbolo y se agrega a la tabla mediante el método agregar().

3.3 Función dentro del Analizador

La integración entre el lexer y la tabla de símbolos se realiza mediante el método registrar() definido en el archivo Definition.flex. Cada vez que se reconoce un patrón, el sistema ejecuta:

$$\text{tabla.agregar}\big(\text{new Simbolo}(\text{yytext}(\quad), \text{tipo}, \text{yyline} + 1, \text{yycolumn} + 1)\big);$$

Esto garantiza que:

- El análisis continúe incluso si existen errores.
- Cada lexema quede documentado.
- Se conserve información precisa de posición.

3.4 Importancia de la Tabla de Símbolos

Aunque en compiladores completos la tabla de símbolos también almacena información semántica (tipo de dato, alcance, valor, etc.), en este laboratorio su función se enfoca en el registro léxico.

Su implementación aporta múltiples beneficios:

- Facilita la visualización del proceso de análisis.
- Permite validar la correcta clasificación de tokens.
- Mejora la detección de errores.
- Sirve como base para futuras extensiones hacia análisis sintáctico.

En conclusión, la tabla de símbolos constituye el componente estructural que conecta el reconocimiento formal de tokens con la representación organizada de la información léxica dentro del sistema.

4. Arquitectura del Proyecto

El proyecto fue estructurado siguiendo la organización estándar de proyectos Java gestionados con Maven, separando claramente el código fuente, la especificación léxica y el código generado automáticamente.

La arquitectura actual del proyecto es la siguiente:

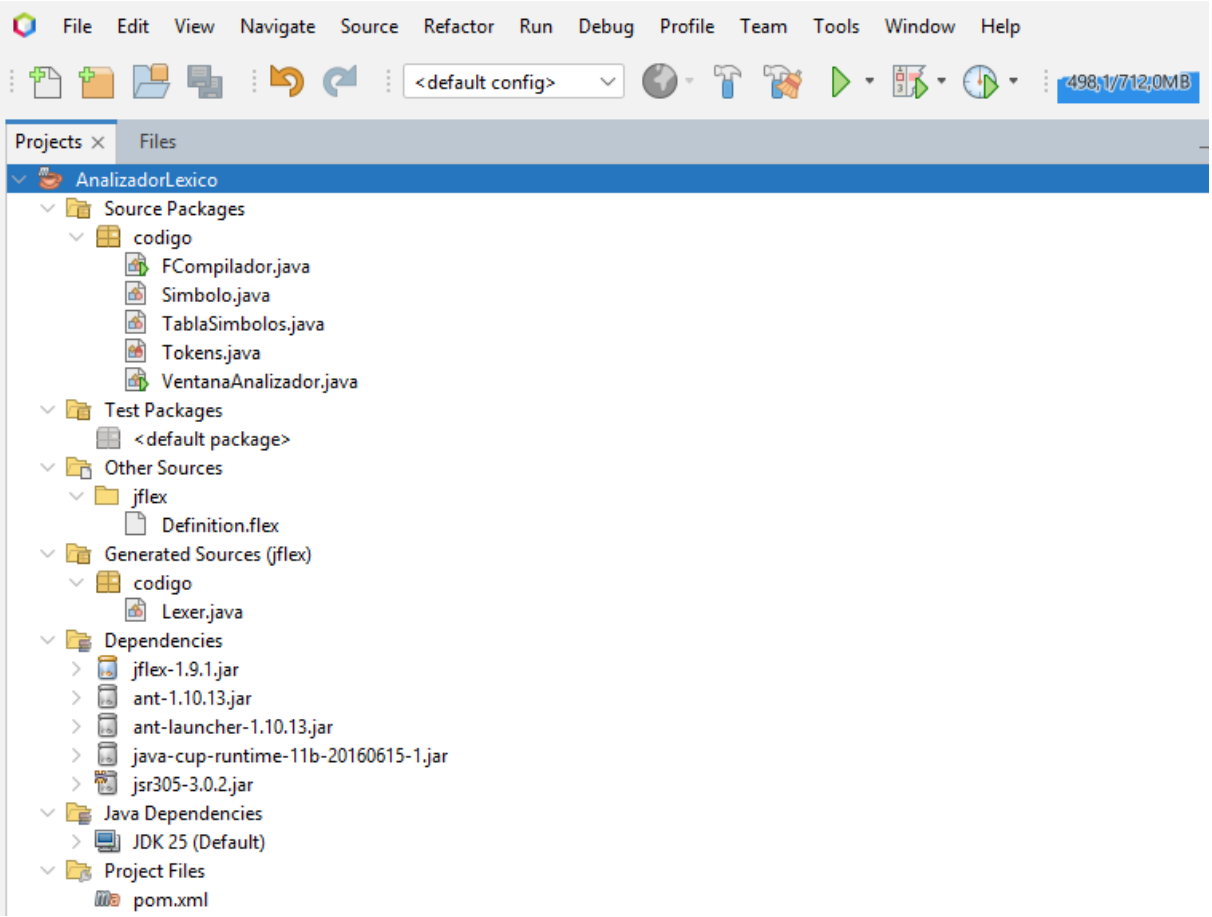

Figura 1. Arquitectura del proyecto en NetBeans

4.1 Paquete código

Este paquete contiene la lógica principal del sistema:

4.1.1 Tokens.java

Define la enumeración de los tipos de tokens reconocidos por el analizador léxico. Cada constante representa una categoría léxica (IDENTIFICADOR, ENTERO, RESERVADA, etc.).

A continuación, la respectiva imagen del código:

```java
package codigo;

/**
 * Enumeración completa de tokens para el analizador léxico de Java.
 * Nivel: Ingeniero — con categorías extendidas para tabla de símbolos.
 */
public enum Tokens {
    RESERVADA,
    IDENTIFICADOR,
    ENTERO,
    DECIMAL,
    CADENA,
    CARACTER,
    OP_ARITMETICO,
    OP_RELACIONAL,
    OP_LOGICO,
    OP_ASIGNACION,
    OP_INCREMENTO,
    OP_DECREMENTO,
    OP_ASIG_COMPUESTO,
    SEPARADOR,
    COMENTARIO_LINEA,
    COMENTARIO_BLOQUE,
    BOOLEANO,
    NULO,
    ERROR
}
```

Figura 2. Enumeración de tipos de tokens

4.1.2 Simbolo.java

Representa una entrada individual dentro de la tabla de símbolos. Contiene:

- Lexema
- Tipo de token
- Línea
- Columna

4.1.3 TablaSimbolos.java

Administra la colección dinámica de símbolos identificados durante el análisis. Permite:

- Registrar tokens
- Registrar errores

- Mostrar la tabla formateada

A continuación, la respectiva imagen del código:

TablaSimbolos.java

Source History

```java
package codigo;

import java.util.ArrayList;
import java.util.List;

/**
 * Tabla de símbolos: almacena todos los tokens encontrados durante
 * el análisis léxico, incluyendo errores con su posición.
 */
public class TablaSimbolos {

    private List<Simbolo> tabla;
    private List<Simbolo> errores;

    public TablaSimbolos() {
        tabla  = new ArrayList<>();
        errores = new ArrayList<>();
    }

    public void agregar(Simbolo s) {
        tabla.add(s);
        if (s.getTipo() == Tokens.ERROR) {
            errores.add(s);
        }
    }

    public List<Simbolo> getTabla()   { return tabla;   }
    public List<Simbolo> getErrores() { return errores; }

    public void imprimirTabla() {
        System.out.println("\n========== TABLA DE SÍMBOLOS ==========");
        System.out.printf("| %-20s | %-20s | %-12s | %-8s |%n",
                "LEXEMA", "TOKEN", "LÍNEA", "COLUMNA");
        System.out.println("-".repeat(75));
        for (Simbolo s : tabla) {
            System.out.println(s);
        }
        System.out.println("=".repeat(75));
        System.out.println("Total tokens: " + tabla.size());
    }

    public void imprimirErrores() {
        if (errores.isEmpty()) {
            System.out.println("\n✅ No se encontraron errores léxicos.");
        } else {
            System.out.println("\n========== ERRORES LÉXICOS ==========");
            for (Simbolo e : errores) {
                System.out.println("❌ ERROR -> " + e);
            }
            System.out.println("Total errores: " + errores.size());
        }
    }
}
```

Figura 3. Implementación de la tabla de símbolos

4.1.4 FCompilador.java

Actúa como núcleo del procesamiento léxico. Coordina la ejecución del lexer y el llenado de la tabla de símbolos.

4.1.5 VentanaAnalizador.java

Implementa la interfaz gráfica del sistema mediante Swing. Permite:

- Ingresar código fuente
- Ejecutar el análisis léxico
- Visualizar los resultados en pantalla

4.2 Carpeta Other Sources / jflex

4.2.1 Definition.flex

Contiene la especificación formal del analizador léxico.
Aquí se definen:

- Directivas del lexer
- Expresiones regulares
- Reglas patrón → acción

Este archivo es la base formal del sistema.

4.3 Carpeta Generated Sources (jflex)

4.3.1 Lexer.java

Archivo generado automáticamente por JFlex durante la compilación del proyecto mediante el plugin jflex-maven-plugin.

Este archivo contiene la implementación del Autómata Finito Determinista (DFA) que reconoce los tokens definidos en Definition.flex.

No debe modificarse manualmente, ya que cualquier cambio será sobrescrito en la siguiente compilación.

4.4 Archivo pom.xml

Define:

- Dependencias (JFlex)
- Plugin de generación automática
- Configuración de compilación
- Clase principal del proyecto

Permite automatizar el proceso de construcción siguiendo prácticas profesionales de desarrollo.

5. Implementación

La implementación del analizador léxico se realizó mediante la herramienta JFlex, utilizando como base el archivo de especificación Definition.flex. Este archivo constituye el núcleo formal del sistema, ya que define los patrones léxicos y las acciones asociadas a cada token.

El archivo Definition.flex se encuentra organizado en tres secciones principales, siguiendo la estructura estándar de JFlex:

5.1 Directivas

En la sección inicial se definen las configuraciones generales del lexer. Entre las directivas utilizadas en el proyecto se encuentran:

- %class Lexer → Define el nombre de la clase generada.
- %unicode → Permite soporte completo de caracteres Unicode.
- %line → Habilita el conteo automático de líneas.
- %column → Habilita el conteo automático de columnas.
- %type Tokens → Especifica que el lexer retornará valores del enum Tokens.

Estas configuraciones permiten que el analizador no solo reconozca patrones, sino que además pueda registrar con precisión la ubicación exacta de cada lexema dentro del código fuente.

5.2 Definiciones (Macros)

La segunda sección del archivo contiene definiciones auxiliares o macros de expresiones regulares. Estas permiten reutilizar patrones complejos de forma más clara y organizada.

Por ejemplo:

- Letras
- Dígitos
- Identificadores
- Espacios en blanco
- Números enteros y decimales

El uso de macros mejora la legibilidad del archivo .flex y evita la repetición innecesaria de expresiones regulares extensas.

5.3 Reglas Léxicas

La sección principal del archivo está compuesta por pares patrón → acción. Cada regla define:

- Una expresión regular que describe el token.
- La acción a ejecutar cuando el patrón es reconocido.

Por ejemplo:

$$\{\text{ENTERO}\}\ \{\ \text{return registrar}(\text{ENTERO});\ \}$$

Cuando el lexer reconoce una secuencia que coincide con la expresión regular de un número entero, ejecuta la acción asociada, registrando el token en la tabla de símbolos.

Cada regla invoca un método registrar() que:

- Crea un nuevo objeto Simbolo.
- Lo agrega a la TablaSimbolos.
- Retorna el tipo de token correspondiente.

A continuación, la imagen respectiva del código:

Definition.flex

Source History

```
40    COM_BLOCK = "/*"([^*]|\*+[^*/])*\*+"/"
41
42    %%
43
44    /* ============ REGLAS ============ */
45
46    /* Palabras reservadas — deben ir ANTES que los identificadores */
47    "abstract"|"assert"|"boolean"|"break"|"byte"|"case"|"catch"|"char"|
48    "class"|"const"|"continue"|"default"|"do"|"double"|"else"|"enum"|
49    "extends"|"final"|"finally"|"float"|"for"|"goto"|"if"|"implements"|
50    "import"|"instanceof"|"int"|"interface"|"long"|"native"|"new"|
51    "package"|"private"|"protected"|"public"|"return"|"short"|"static"|
52    "strictfp"|"super"|"switch"|"synchronized"|"this"|"throw"|"throws"|
53    "transient"|"try"|"void"|"volatile"|"while"
54        { return registrar(RESERVADA); }
55
56    /* Literales booleanos y nulo */
57    "true"|"false"    { return registrar(BOOLEANO); }
58    "null"            { return registrar(NULO); }
59
60    /* Identificadores */
61    {ID}              { return registrar(IDENTIFICADOR); }
62
63    /* Números */
64    {DECIMAL}         { return registrar(DECIMAL); }
65    {ENTERO}          { return registrar(ENTERO); }
66
67    /* Cadenas y caracteres */
68    {CADENA}          { return registrar(CADENA); }
69    {CARACTER}        { return registrar(CARACTER); }
70
71    /* Comentarios */
72    {COM_LINE}        { return registrar(COMENTARIO_LINEA); }
73    {COM_BLOCK}       { return registrar(COMENTARIO_BLOQUE); }
74
75    /* Operadores de asignación compuesta */
76    "+=" | "-=" | "*=" | "/=" | "%="
77        { return registrar(OP_ASIG_COMPUESTO); }
78
79    /* Incremento y decremento */
80    "++"              { return registrar(OP_INCREMENTO); }
81    "--"              { return registrar(OP_DECREMENTO); }
82
83    /* Operadores relacionales */
84    "==" | "!=" | ">=" | "<=" | ">" | "<"
85        { return registrar(OP_RELACIONAL); }
86
87    /* Operadores lógicos */
88    "&&" | "||" | "!"
89        { return registrar(OP_LOGICO); }
90
91    /* Operadores aritméticos */
92    "+" | "-" | "*" | "/" | "%"
93        { return registrar(OP_ARITMETICO); }
94
95    /* Operador de asignación */
96    "="               { return registrar(OP_ASIGNACION); }
97
98    /* Separadores */
99    "(" | ")" | "{" | "}" | "[" | "]" | ";" | "," | "." | ":"
100       { return registrar(SEPARADOR); }
101
102   /* Espacios en blanco — se ignoran */
103   {ESPACIO}         { /* ignorar */ }
104
105   /* Cualquier otro carácter es un ERROR */
106   .                 { return registrar(ERROR); }
107
```

Figura 4. Especificación léxica en Definition.flex

5.4 Manejo de Errores Léxicos

Una característica importante del sistema implementado es que el manejo de errores no detiene la ejecución del análisis.

Se definió una regla final en el archivo .flex para capturar cualquier carácter no reconocido:

$$. \ \{ \text{return registrar}(\text{ERROR}); \}$$

Esto garantiza que:

- El analizador continúe procesando el resto del código.
- Todos los errores sean registrados.
- Se genere un reporte completo en una sola ejecución.

Durante la ejecución, los errores se almacenan en una lista independiente dentro de TablaSimbolos, permitiendo presentar al usuario un resumen final con el número total de errores léxicos detectados.

5.5 Integración con Maven

La generación del archivo Lexer.java se automatiza mediante el plugin:

$$\text{jflex} - \text{maven} - \text{plugin}$$

Cada vez que el proyecto se compila:

- Maven ejecuta JFlex.
- Se genera automáticamente el archivo Lexer.java.
- El archivo se coloca en:

$$\text{target/generated} - \text{sources/jflex/}$$

- El código se compila junto con el resto del proyecto.

Esto garantiza coherencia entre la especificación formal y el código ejecutable, siguiendo prácticas profesionales de automatización.

5.6 Integración con la Interfaz Gráfica

La clase VentanaAnalizador permite interactuar con el sistema de forma visual. El flujo de ejecución es el siguiente:

El usuario ingresa código fuente en el área de texto de entrada.

- Presiona el botón Analizar.
- Se instancia el lexer generado por JFlex.
- Se procesa el texto carácter por carácter.
- Se llena la tabla de símbolos.
- Se muestra el resultado formateado en el área de salida.

Esta integración demuestra la conexión entre la teoría formal del análisis léxico y su aplicación práctica dentro de una interfaz gráfica funcional.

6. Ejecución y Resultados

Para validar el funcionamiento del analizador léxico implementado, se ejecutó el sistema sobre un fragmento de código Java que contiene declaraciones de variables, operadores aritméticos, operadores relacionales, estructuras condicionales, comentarios y errores léxicos intencionales.

6.1 Código Analizado

El siguiente código fue ingresado en el área de texto de la interfaz gráfica:

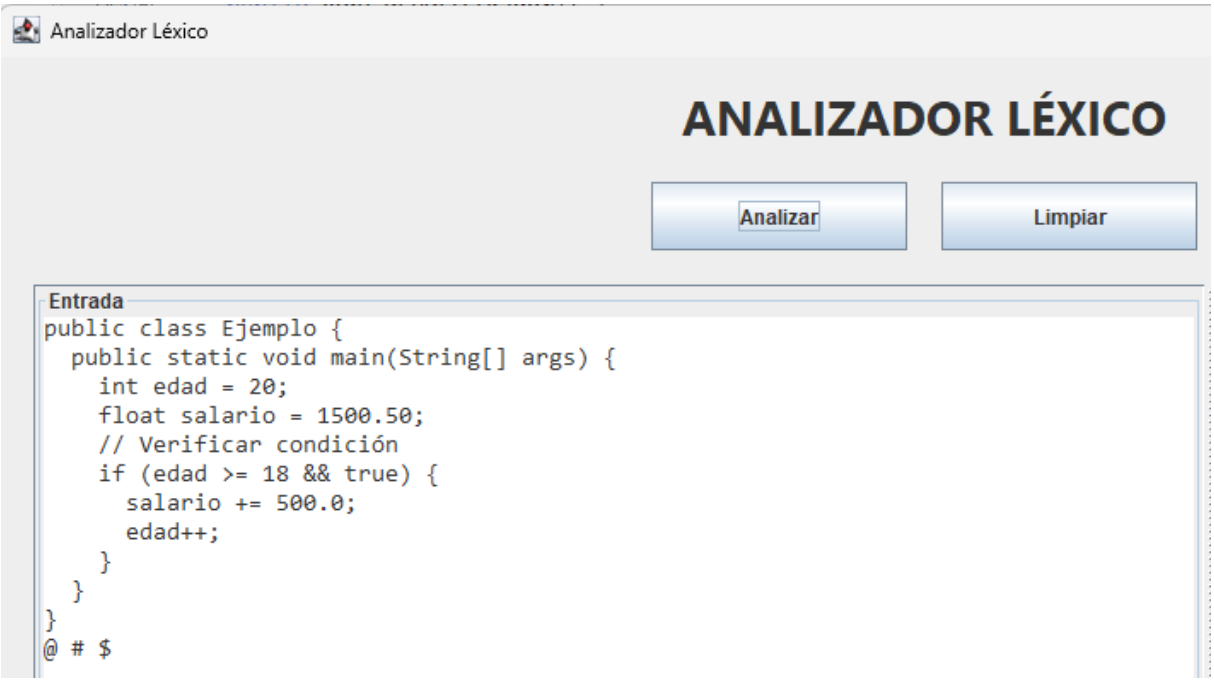

Figura 5. Interfaz gráfica del analizador léxico en ejecución

Este fragmento fue diseñado para evaluar múltiples categorías léxicas:

- Palabras reservadas (public, class, if, else)
- Identificadores (edad, salario, activo)
- Literales numéricos (20, 1500.50)
- Cadenas de texto ("Juan")
- Operadores (=, +=, ++, >=, &&)
- Separadores ({}, (), ;)
- Comentarios de línea y bloque
- Caracteres inválidos (@, #, $)

6.2 Resultados del Análisis

Al presionar el botón Analizar, el sistema procesó el texto carácter por carácter utilizando el DFA generado por JFlex. El resultado fue mostrado en el área de salida en forma de tabla estructurada.

El sistema identificó correctamente:

- 71 tokens válidos
- 3 errores léxicos

Cada token fue registrado con la siguiente información:

- Lexema
- Tipo de token
- Línea
- Columna

Ejemplo de salida parcial:

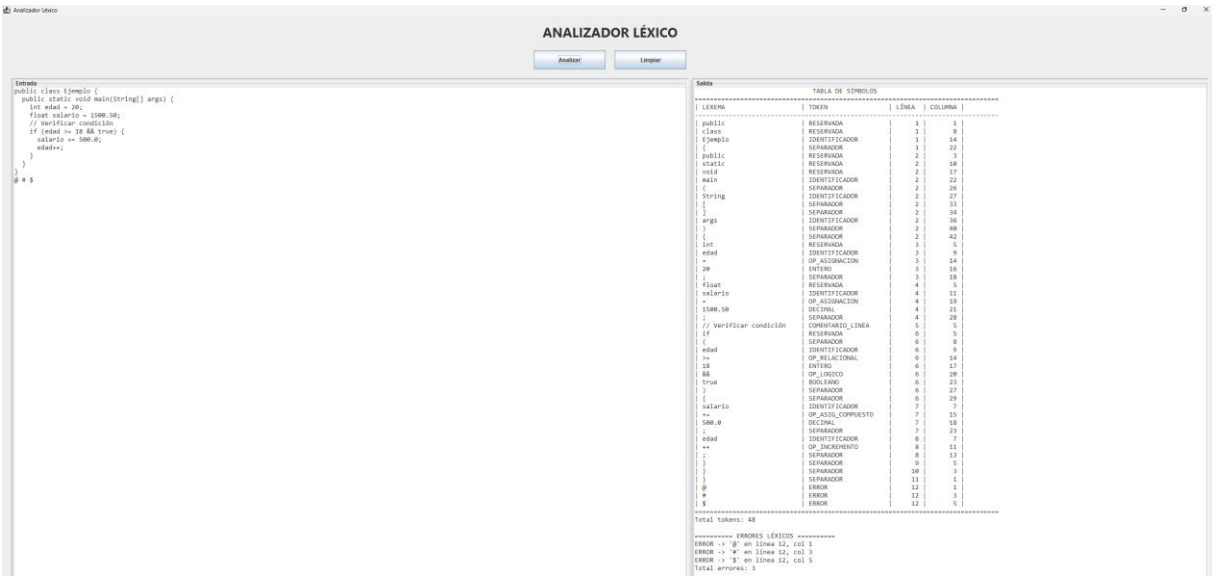

Figura 6. Detección de errores léxicos

6.3 Detección de Errores Léxicos

Los caracteres:

@ # $

no corresponden a ningún patrón definido en el archivo Definition.flex. Por lo tanto, fueron clasificados como:

ERROR

El sistema no detuvo la ejecución al encontrar estos caracteres, sino que los registró y continuó el análisis, mostrando el siguiente resumen final:

- Total tokens: 71
- Total errores: 3

Este comportamiento demuestra la correcta implementación del manejo robusto de errores léxicos.

6.4 Validación del Funcionamiento

Los resultados obtenidos confirman que:

- Las expresiones regulares fueron correctamente definidas.
- El DFA generado por JFlex reconoce adecuadamente los patrones.
- La tabla de símbolos almacena correctamente cada entrada.
- El sistema mantiene la ejecución aun en presencia de errores.
- La interfaz gráfica permite visualizar claramente el proceso de análisis.

En conclusión, la ejecución del laboratorio evidencia la correcta integración entre teoría formal (autómatas y expresiones regulares), automatización mediante JFlex y aplicación práctica en un entorno visual funcional.

Y con este ultimo punto se da por finalizado el desarrollo de este laboratorio.

¡¡¡Mil gracias respetado profesor.!!!

CONCLUSIONES DE LA ACTIVIDAD

Para concluir este trabajo, se puede afirmar que el desarrollo del laboratorio permitió comprender de manera práctica el funcionamiento del análisis léxico como primera fase en la construcción de compiladores. A través del uso de JFlex como generador automático de analizadores léxicos, se evidenció cómo los fundamentos teóricos de las expresiones regulares y los autómatas finitos deterministas se transforman en una solución computacional funcional y eficiente.

A diferencia de un enfoque meramente teórico, en el cual conceptos como tokens, expresiones regulares o DFA se estudian de forma abstracta, la implementación práctica permitió consolidar el conocimiento mediante la construcción real de un analizador léxico. El proceso inició con la definición formal de los tokens en el archivo Definition.flex, continuó con la generación automática del archivo Lexer.java mediante Maven y culminó con la integración del sistema en una interfaz gráfica que permitió visualizar el proceso de análisis.

Durante la ejecución del sistema se observó que el analizador identifica correctamente palabras reservadas, identificadores, literales numéricos, operadores, separadores y comentarios, registrando cada elemento en la tabla de símbolos junto con su línea y columna correspondiente. Esto demuestra la correcta implementación de las expresiones regulares y la adecuada generación del autómata por parte de JFlex.

Un aspecto especialmente relevante evidenciado en este laboratorio es la importancia del manejo de errores léxicos. El sistema no detiene su ejecución ante la presencia de caracteres no reconocidos, sino que los registra como errores y continúa el procesamiento del código fuente. Este comportamiento refleja un diseño robusto, alineado con prácticas reales en la construcción de compiladores.

Asimismo, la implementación de la tabla de símbolos permitió comprender la diferencia entre la definición formal de tokens y su representación estructurada durante la ejecución. Mientras la especificación léxica define qué patrones pueden ser reconocidos, la tabla de símbolos almacena la información concreta de cada lexema identificado, constituyendo el puente entre el reconocimiento formal y la representación organizada del análisis.

La integración del proyecto con Maven y el uso del plugin jflex-maven-plugin demuestran la importancia de la automatización en el desarrollo profesional. La generación automática del lexer garantiza coherencia entre la especificación formal y el código ejecutable, reduciendo errores manuales y mejorando la mantenibilidad del sistema.

En síntesis, este laboratorio permitió aplicar de manera práctica los fundamentos de la teoría de autómatas y lenguajes formales, consolidando competencias en:

- Diseño de expresiones regulares.
- Construcción formal de analizadores léxicos.
- Implementación de tablas de símbolos.
- Manejo estructurado de errores.
- Automatización del proceso de generación de código.

En conclusión, el trabajo evidencia cómo los principios matemáticos de los autómatas finitos deterministas se materializan en herramientas reales de desarrollo, demostrando que el análisis léxico no es únicamente un concepto teórico, sino una etapa esencial y estructural dentro del proceso de construcción de compiladores y sistemas de procesamiento de lenguajes.

BIBLIOGRAFÍA

A continuación, la bibliografía implementada en este desarrollo:

- Tema 1. Minería de texto. Procesadores de Lenguajes (COLGII) - PER 15746 - Enero 2026.
- Tema 2. Procesamiento del lenguaje natural. Procesadores de Lenguajes (COLGII) - PER 15746 - Enero 2026.
- Tema 3. Introducción al proceso de compilación. Procesadores de Lenguajes (COLGII) - PER 15746 - Enero 2026.

- Tema 4. Analizador léxico I. Procesadores de Lenguajes (COLGII) - PER 15746 - Enero 2026.
- Tema 5. Analizador léxico II. Procesadores de Lenguajes (COLGII) - PER 15746 - Enero 2026.
- Clases virtuales con el profesor Ing. Rogerio Orlando Beltrán Castro.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Pearson.
- Klein, G. (2023). JFlex – The Fast Scanner Generator for Java. Recuperado de https://jflex.de
- Oracle Corporation. (2024). The Java Language Specification, Java SE 21 Edition. Oracle. https://docs.oracle.com/javase/specs/
- Maven Project. (2024). Apache Maven Project Documentation. The Apache Software Foundation. https://maven.apache.org

AGRADECIMIENTO

Finalmente, deseo expresar mi más sincero agradecimiento al profesor Ing. Rogerio Orlando Beltrán Castro, por los conocimientos, orientación y acompañamiento brindados durante el desarrollo de esta actividad. Sus explicaciones en el área de compiladores, teoría de autómatas y análisis de lenguajes de programación fueron fundamentales para comprender los principios formales del análisis léxico y su aplicación práctica mediante herramientas especializadas.

Gracias a los conceptos compartidos en clase, fue posible desarrollar este laboratorio de manera estructurada, aplicando correctamente la definición formal de tokens, el uso de expresiones regulares, la comprensión del funcionamiento de los Autómatas Finitos Deterministas (DFA) y la generación automática de analizadores léxicos mediante JFlex. La guía metodológica proporcionada permitió abordar el proceso paso a paso, garantizando una comprensión clara de la transición entre teoría formal e implementación práctica.

Asimismo, el acompañamiento académico facilitó la integración de herramientas profesionales como Maven y la construcción de una interfaz gráfica que permitió visualizar el funcionamiento del analizador léxico en tiempo real. Esta orientación fue clave para consolidar una visión integral del proceso de compilación desde su primera fase.

Sin duda, esta experiencia fortaleció significativamente mi formación académica, ampliando mi comprensión sobre cómo los fundamentos matemáticos de los lenguajes formales y los autómatas se materializan en sistemas computacionales reales. Este laboratorio no solo consolidó los conceptos teóricos estudiados, sino que también permitió evidenciar la relevancia práctica del análisis léxico dentro del campo de la ingeniería informática.

¡¡¡Mil gracias profesor!!!

Respetuosamente,

Alejandro De Mendoza